Submitted by

Y. Divya Sree

192472357

CSA09 – PROGRAMMING IN JAVA

A JAVA-BASED WAREHOUSE MANAGEMENT SYSTEM USING OBJECT-ORIENTED PROGRAMMING CONCEPTS

A. ASSIGNMENT INFORMATION

Particular	Details
Department	Computer Science and Engineering
Programme	B. Tech – Computer Science and Engineering
Course Code	CSA0923
Course Name	Programming in JAVA
Academic Year / Batch	2026–2027
Assignment Title	A Java-based Warehouse Management System using Object-Oriented Programming Concepts
Date of Issue	01-09-26
Date of Submission	03-09-26
Maximum Marks	100
CO	CO1, CO2, CO3
Bloom's Level	L3 – Apply, L4 – Analyse
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Smart warehousing, logistics automation and inventory management

The assignment focuses on applying Java programming and object-oriented programming concepts to design an autonomous warehouse management system. The required learning outcomes include OOP principles, Java Collection Framework, and multithreaded programming.

B. ASSIGNMENT PROBLEM / CHALLENGE

Modern warehouses handle large numbers of products, storage locations, robots and operational tasks. Manual management of these resources can lead to delays, incorrect inventory information and inefficient task allocation.

The challenge is to develop a Java-based **Autonomous Warehouse Management System** that represents warehouse entities as objects and allows robots to perform different tasks automatically.

The system should demonstrate:

- Classes and objects
- Encapsulation and data hiding
- Inheritance
- Polymorphism and method overriding
- Java Collection Framework
- Generics and iterators
- Exception handling
- Multithreading
- Thread priorities
- Synchronization
- Inter-thread communication

The architecture should also be extensible so that new robot types and warehouse operations can be added without redesigning the complete system.

C. PROBLEM STATEMENT

Design and implement an object-oriented Java architecture for an **Autonomous Warehouse Management System**.

The system contains entities such as:

- Robots
- Warehouse items
- Storage locations
- Warehouse tasks

A common abstract Robot class is created for general robot properties and operations. Specialized robots such as Transport Robot, Picking Robot, and Inventory Robot inherit from the common class.

Encapsulation is used to protect important data such as battery level, robot status and item information. Inheritance provides code reuse, while polymorphism allows different robot types to perform common tasks differently.

Java collections are used to store and retrieve groups of objects. Multithreading is used to process warehouse tasks concurrently. Exception handling is included to manage invalid operations such as insufficient battery, unavailable items and invalid locations.

The system is designed to be modular, reusable and extendable.

D. REQUIREMENTS AND CONSTRAINTS

D.1 Functional Requirements

Requirement ID	Requirement
FR1	Register robots in the warehouse
FR2	Store item information
FR3	Maintain warehouse locations
FR4	Create and assign tasks
FR5	Allow different robots to perform tasks
FR6	Update robot battery and status
FR7	Maintain collections of robots, items and tasks
FR8	Process tasks using threads
FR9	Handle invalid operations using exceptions
FR10	Display warehouse operation results

D.2 Technical Requirements

- Java should be used for implementation.
- OOP principles must be applied.
- Generic collections should be used.
- Iterators should be demonstrated.
- Exception handling must be included.
- Multithreading should be implemented.
- Synchronization should prevent conflicting task execution.

D.3 Constraints

1. Robot battery level must remain between 0 and 100%.
2. A robot cannot perform a task when its battery is insufficient.
3. Each item must have a unique item ID.
4. Locations must have unique IDs.

5. Only registered robots can receive tasks.
6. Shared warehouse resources must be synchronized.
7. The system should support future robot types.

E. STUDENT WORK

E.1 PROBLEM UNDERSTANDING AND FORMULATION

E.1.1 Problem Understanding

An autonomous warehouse contains multiple robots that perform different operations. For example, a transport robot can move items, a picking robot can collect items, and an inventory robot can inspect stock.

Although these robots have different responsibilities, they share common properties such as:

- Robot ID
- Battery level
- Current location
- Robot status

Therefore, a common superclass can represent the shared properties, while subclasses implement robot-specific behaviour.

The warehouse also maintains items, locations and tasks. These objects interact with each other during task execution.

E.1.2 Expected Outcomes

The proposed system should:

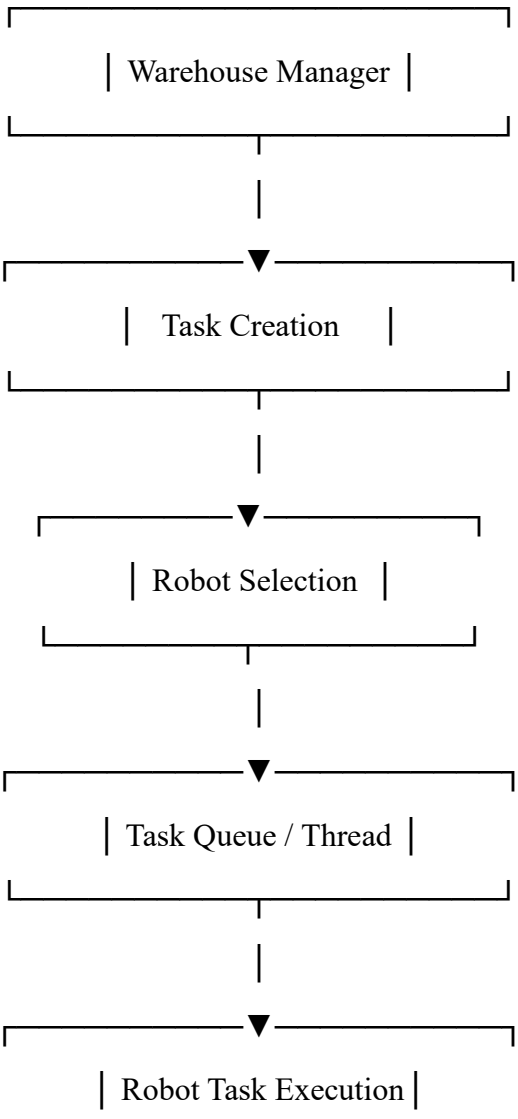
1. Model warehouse entities using Java classes.
2. Apply encapsulation to protect object data.
3. Use inheritance for specialized robots.
4. Demonstrate polymorphism through overridden methods.
5. Store objects using Java collections.
6. Process warehouse tasks concurrently.
7. Handle errors safely.
8. Provide an extendable warehouse architecture.

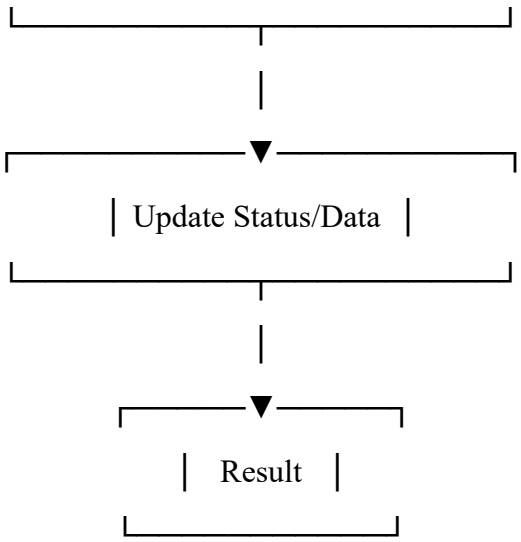
E.1.3 Major Entities

Entity	Purpose
Robot	Represents common robot properties
Transport Robot	Moves warehouse items
Picking Robot	Performs item-picking operations
Inventory Robot	Performs inventory checking
Item	Represents warehouse products
Location	Represents storage position
Task	Represents an operation assigned to a robot
Warehouse Manager	Controls warehouse objects and operations

E.1.4 SYSTEM WORKFLOW

Figure 1: Overall Warehouse Management Workflow





E.2 APPLICATION OF COURSE KNOWLEDGE

E.2.1 Object-Oriented Design

The system uses the following OOP principles:

1. Classes and Objects

Each warehouse entity is represented using a class.

Example:

Item item1 = new Item ("I101", "Laptop", 20);

Here, Item is the class and item1 is an object.

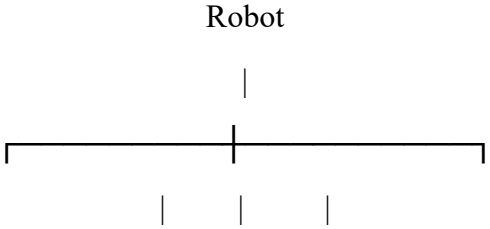
2. Encapsulation

Data members are declared as private and accessed through controlled methods.

```
private double battery Level;
public double getBatteryLevel () {
   return battery Level;
}
```

3. Inheritance

Specialized robot classes inherit common properties from Robot.



Transport Robot | Picking Robot | Inventory Robot

Figure 2: Robot Inheritance Hierarchy

4. Polymorphism

A common reference can point to different robot objects.

```
Robot r;
r = new TransportRobot(...);
r.performTask(task);
r = new PickingRobot(...);
r.performTask(task);
```

The appropriate overridden method is selected at runtime.

5. Abstraction

The Robot class is declared abstract because it represents common robot behaviour but does not define one fixed implementation for every task.

E.2.2 Java Collection Framework

Collections are used to efficiently manage multiple warehouse objects.

Collection	**Purpose**
Array List<Robot>	Store registered robots
Array List<Task>	Store warehouse tasks
HashMap<String, Item>	Retrieve items using item ID
HashSet<Location>	Maintain unique locations
Iterator<Robot>	Traverse robot objects
Generics	Provide type safety

Example:

```
Array List<Robot> robots = new Array List<>();
HashMap<String, Item> items = new HashMap<>();
HashSet<Location> locations = new HashSet<>();
```

E.2.3 Exception Handling

Warehouse operations may fail because of invalid input or insufficient resources.

A custom exception is used:

```
class Warehouse Exception extends Exception {
   public Warehouse Exception(String message) {
      super(message);
   }
}
```

Example:

```
if (battery Level < 20) {
   throw new Warehouse Exception("Insufficient battery");
}
```

This prevents the program from terminating unexpectedly.

E.2.4 Multithreading

Warehouse robots may operate simultaneously. Java threads are used to represent task-processing activities.

The system demonstrates:

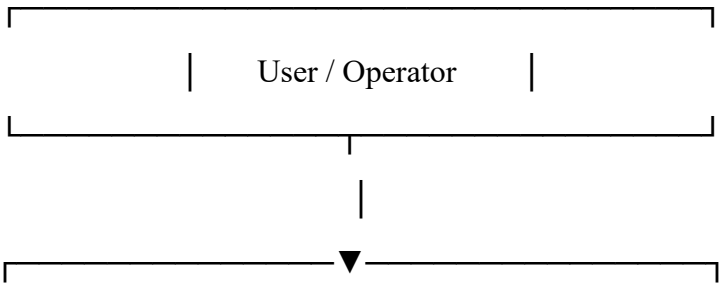
- Thread creation
- Thread priority
- Synchronization
- Inter-thread communication
- Exception handling in threads

A task-processing thread can process tasks while another operation continues independently.

E.3 SOLUTION / DESIGN / METHODOLOGY

E.3.1 Proposed Architecture

The system follows a simple layered object-oriented architecture.



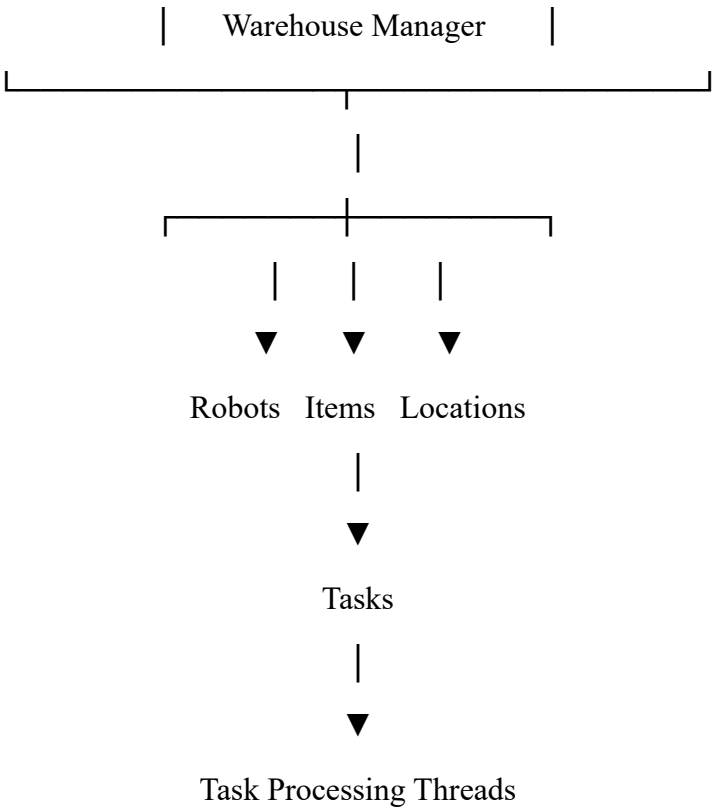


Figure 3: Proposed System Architecture

E.3.2 Class Design

Class	**Main Attributes**	**Main Methods**
Robot	id, battery, status	perform Task (), charge ()
Transport Robot	inherited data	perform Task ()
Picking Robot	inherited data	perform Task()
Inventory Robot	inherited data	perform Task()
Item	id, name, quantity	updateQuantity()
Location	id, zone	displayLocation()
Task	id, type, item	getTaskType()
Warehouse Manager	collections	addRobot(), addItem(), assignTask()
WarehouseException	message	exception handling

E.3.3 PSEUDOCODE

START

Create WarehouseManager

Create robots:

TransportRobot

PickingRobot

InventoryRobot

Create warehouse items

Create storage locations

Add robots, items and locations to collections

Create warehouse task

FOR each task

Select suitable robot

Check robot battery

IF battery is insufficient

Generate Warehouse Exception

ELSE

Assign task to robot

Execute robot task

Update battery

Update robot status

END IF

END FOR

Create task-processing threads

Set thread priorities

Synchronize shared warehouse operations

Display robot status and task results

STOP

E.3.4 JAVA IMPLEMENTATION

The following implementation demonstrates the major concepts required by the assignment.

```
import java.util.*;
class WarehouseException extends Exception {
    public WarehouseException(String message) {
```

```
        super(message);
    }
}

class Item {
    private String id;
    private String name;
    private int quantity;
    public Item(String id, String name, int quantity) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
    }
    public String getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public int getQuantity() {
        return quantity;
    }
    public void updateQuantity(int q) {
        quantity += q;
    }
    public String toString() {
        return id + " - " + name + " - Qty: " + quantity;
    }
}
```

```
class Location {
    private String id;
    private String zone;

    public Location(String id, String zone) {
        this.id = id;
        this.zone = zone;
    }
    public String getId() {
        return id;
    }
    public String toString() {
        return id + " (" + zone + ")";
    }
    public boolean equals(Object o) {
        if (!(o instanceof Location))
            return false;
        Location l = (Location)o;
        return id.equals(l.id);
    }
    public int hashCode() {
        return id.hashCode();
    }
}
class Task {
    private String id;
    private String type;
    private Item item;
```

```
    public Task(String id, String type, Item item) {
        this.id = id;
        this.type = type;
        this.item = item;
    }
    public String getType() {
        return type;
    }
    public Item getItem() {
        return item;
    }
    public String toString() {
        return id + " - " + type;
    }
}
abstract class Robot {
    private String id;
    private double battery;
    private String status;
    public Robot(String id, double battery) {
        this.id = id;
        this.battery = battery;
        this.status = "IDLE";
    }
    public String getId() {
        return id;
    }
    public double getBattery() {
        return battery;
```

```
    }
    public String getStatus() {
        return status;
    }

    protected void setStatus(String status) {
        this.status = status;
    }
    protected void useBattery(double value)
            throws WarehouseException {
        if (battery < value)
            throw new WarehouseException(
                "Insufficient battery for robot " + id)
        battery -= value;
    }
    public void charge() {
        battery = 100;
        status = "IDLE";
    }
    public abstract void performTask(Task task)
            throws WarehouseException;

    public String toString() {
        return id + " | Battery: " +
                battery + "% | Status: " + status;
    }
}
class TransportRobot extends Robot {
    public TransportRobot(String id, double battery) {
```

```
        super(id, battery);
    }
    @Override
    public void performTask(Task task)
            throws WarehouseException {
        setStatus("TRANSPORTING");
        useBattery(15);
        System.out.println(
            getId() + " transported " +
            task.getItem().getName());
        setStatus("IDLE");
    }
}
class PickingRobot extends Robot {
    public PickingRobot(String id, double battery) {
        super(id, battery);
    }
    @Override
    public void performTask(Task task)
            throws WarehouseException {
        setStatus("PICKING");
        useBattery(10);
        if (task.getItem().getQuantity() <= 0)
            throw new WarehouseException("Item unavailable");
        task.getItem().updateQuantity(-1);
        System.out.println(
            getId() + " picked " +
            task.getItem().getName());
        setStatus("IDLE");
```

```
    }
}
class InventoryRobot extends Robot {

    public InventoryRobot(String id, double battery) {
        super(id, battery);
    }
    @Override
    public void performTask(Task task)
            throws WarehouseException {
        setStatus("CHECKING");
        useBattery(5);
        System.out.println(
            getId() + " checked inventory of " +
            task.getItem().getName());
        setStatus("IDLE");
    }
}
class WarehouseManager {
    private ArrayList<Robot> robots = new ArrayList<>();
    private ArrayList<Task> tasks = new ArrayList<>();
    private HashMap<String, Item> items =
            new HashMap<>();
    private HashSet<Location> locations =
            new HashSet<>();
    public synchronized void addRobot(Robot r) {
        robots.add(r);
    }
    public synchronized void addItem(Item i) {
```

```
    items.put(i.getId(), i);
}
public synchronized void addLocation(Location l) {
    locations.add(l);
}
public synchronized void addTask(Task t) {
    tasks.add(t);
}
public void displayRobots() {
    Iterator<Robot> it = robots.iterator();
    while (it.hasNext()) {
        System.out.println(it.next());
    }
}
public void executeTasks() {
    for (Task t : tasks) {
        for (Robot r : robots) {
            try {
                if (t.getType().equals("TRANSPORT")
                        && r instanceof TransportRobot) {
                    r.performTask(t);
                    break;
                } else if (t.getType().equals("PICK")
                        && r instanceof PickingRobot) {
                    r.performTask(t);
                    break;
                } else if (t.getType().equals("INVENTORY")
                        && r instanceof InventoryRobot) {
                    r.performTask(t);
```

```
                    break;
                }
            } catch (WarehouseException e) {
                System.out.println(
                    "Error: " + e.getMessage());
            }
        }
    }
}
class TaskProcessor extends Thread {
    private WarehouseManager manager;
    public TaskProcessor(WarehouseManager manager) {
        this.manager = manager;
    }
    public void run() {
        synchronized (manager) {
            manager.executeTasks();
            manager.notifyAll();
        }
    }
}
public class WarehouseSystem {
    public static void main(String[] args)
            throws InterruptedException {
        WarehouseManager manager =
                new WarehouseManager();
        Item i1 =
            new Item("I101", "Laptop", 10);
```

```
Item i2 =
    new Item("I102", "Keyboard", 15);
manager.addItem(i1);
manager.addItem(i2);

manager.addLocation(
    new Location("L01", "Storage-A"));
manager.addLocation(
    new Location("L02", "Storage-B"));
Robot r1 =
    new TransportRobot("R01", 80);
Robot r2 =
    new PickingRobot("R02", 90);
Robot r3 =
    new InventoryRobot("R03", 70);
manager.addRobot(r1);
manager.addRobot(r2);
manager.addRobot(r3);
manager.addTask(
    new Task("T01", "TRANSPORT", i1));
manager.addTask(
    new Task("T02", "PICK", i2));
manager.addTask(
    new Task("T03", "INVENTORY", i1));
TaskProcessor processor =
    new TaskProcessor(manager);
processor.setPriority(Thread.MAX_PRIORITY);
processor.start();
processor.join()
```

```
        System.out.println("\nFinal Robot Status:");
        manager.displayRobots();
        System.out.println("\nInventory:");
        System.out.println(i1);
        System.out.println(i2);
    }
}
```

E.3.5 POLYMORPHIC TASK EXECUTION

The most important polymorphic operation is:

Robot r;

The reference can represent:

r = new TransportRobot(...);

r.performTask(task);

or:

r = new PickingRobot(...);

r.performTask(task);

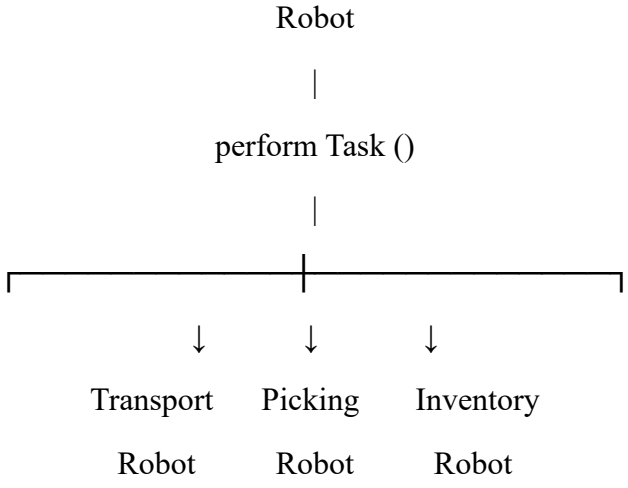
or:

r = new InventoryRobot(...);

r.performTask(task);

Although the method call is the same, the implementation depends on the actual object type.

Figure 4: Polymorphic Robot Task Execution



↓ ↓ ↓

Transport Item | Pick Item | Check Stock

E.3.6 TASK MAPPING

Robot Type	Task	Operation
TransportRobot	TRANSPORT	Move item
PickingRobot	PICK	Pick item
InventoryRobot	INVENTORY	Check stock

E.3.7 MULTITHREADING DESIGN

The TaskProcessor class extends Thread.

```
class TaskProcessor extends Thread {
   public void run() {
      ...
   }
}
```

Thread priority is assigned using:

```
processor.setPriority(Thread.MAX_PRIORITY);
```

Synchronization is applied to shared warehouse operations:

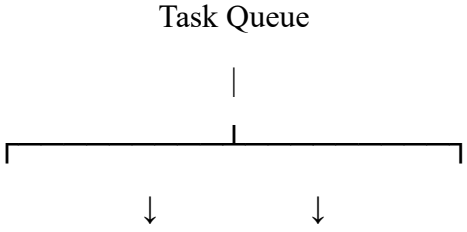
```
public synchronized void addRobot(Robot r) {
   robots.add(r);
}
```

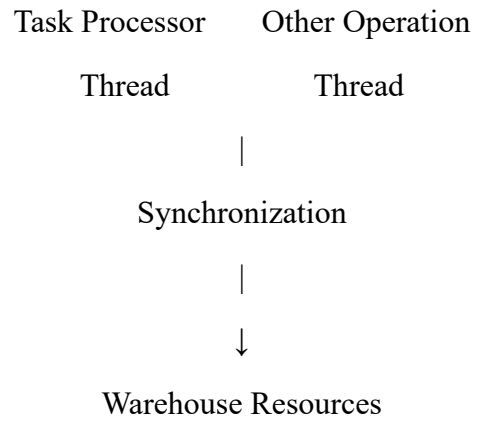
Inter-thread communication is demonstrated using:

```
manager.notifyAll();
```

This design prevents multiple threads from modifying shared warehouse data at the same time.

Figure 5: Multithreaded Task Processing





E.4 USE OF MODERN TOOLS

Tool / Technology	**Purpose**
Java JDK	Program development and execution
IntelliJ IDEA / Eclipse / VS Code	Source-code development
Git	Version control
GitHub	Code repository and submission
Java Collection Framework	Object storage and retrieval
Java Threads	Concurrent task processing

Development Procedure

1. Create Java project.
2. Create warehouse classes.
3. Implement OOP relationships.
4. Add collection framework.
5. Add exception handling.
6. Implement task-processing thread.
7. Compile and test the program.
8. Upload source code to GitHub.

The assignment deliverables specifically require pseudocode, implementation/results and GitHub upload.

E.5 RESULTS AND VALIDATION

E.5.1 Initial Test Data

Robot ID	**Type**	**Battery**

R01	TransportRobot	80%
R02	PickingRobot	90%
R03	InventoryRobot	70%

Item ID	Item	Quantity
I101	Laptop	10
I102	Keyboard	15

Task ID	Type	Item
T01	TRANSPORT	Laptop
T02	PICK	Keyboard
T03	INVENTORY	Laptop

E.5.2 Sample / Expected Output

The following is the expected output based on the sample input:

R01 transported Laptop

R02 picked Keyboard

R03 checked inventory of Laptop

Final Robot Status:

R01 | Battery: 65.0% | Status: IDLE

R02 | Battery: 80.0% | Status: IDLE

R03 | Battery: 65.0% | Status: IDLE

Inventory:

I101 - Laptop - Qty: 10

I102 - Keyboard - Qty: 14

E.5.3 Test Cases

Test ID	Test Condition	Expected Result
TC01	Add valid robot	Robot registered
TC02	Add item	Item stored in HashMap
TC03	Execute transport task	Item transport completed
TC04	Execute picking task	Quantity decreases

TC05	Execute inventory task	Inventory displayed
TC06	Battery below requirement	Exception generated
TC07	Empty item quantity	Item unavailable exception
TC08	Duplicate location	HashSet avoids duplicate
TC09	Multiple task operations	Tasks processed successfully
TC10	Shared resource access	Synchronization protects data

E.5.4 Requirement Validation

Requirement	Validation Result
OOP classes and objects	Achieved
Encapsulation	Achieved
Inheritance	Achieved
Polymorphism	Achieved
Collections	Achieved
Generics	Achieved
Iterator	Achieved
Exception handling	Achieved
Multithreading	Achieved
Synchronization	Achieved
Thread priority	Achieved
Extensibility	Achieved

E.6 ANALYSIS AND ENGINEERING DECISION

E.6.1 Design Alternatives

Approach	Advantages	Limitations
Single large class	Easy initial coding	Poor maintainability
Separate independent classes	Better organization	More coordination required
OOP inheritance-based design	Reusable and extensible	Requires proper class hierarchy

Proposed OOP architecture	Reusable, modular and scalable	Slightly more design effort

The proposed OOP architecture is selected because it provides a balance between simplicity and extensibility.

E.6.2 Advantages

- Reduces code duplication.
- Protects important data.
- Supports different robot behaviours.
- Makes object interaction clear.
- Allows efficient collection management.
- Supports concurrent operations.
- Makes future robot types easier to add.

E.6.3 Limitations

The prototype does not include:

- Real physical robots
- Sensors
- Real-time location tracking
- Database connectivity
- Graphical user interface
- Real warehouse hardware integration

These can be included in future versions.

E.6.4 Extensibility

A new robot can be added by extending the Robot class.

Example:

```
class CleaningRobot extends Robot {k
   @Override
   public void performTask(Task task)
         throws WarehouseException {    }
}
```

Existing robot classes do not need to be modified. This demonstrates the extensibility advantage of inheritance and polymorphism.

E.7 BROADER CONSIDERATIONS

E.7.1 Sustainability

Automation can reduce unnecessary movement and improve warehouse resource utilization. Efficient task assignment can also reduce energy consumption of warehouse robots.

E.7.2 Societal Relevance

Automated warehouse systems can improve order processing, inventory accuracy and operational safety. They can reduce repetitive manual work and support modern logistics industries.

E.7.3 Safety

Battery validation and exception handling help prevent unsafe robot operations. Synchronization also reduces the possibility of conflicting access to shared resources.

E.7.4 Ethical Considerations

The system should be designed so that automation improves productivity while considering worker safety, accessibility and responsible use of technology.

E.7.5 SDG Relevance

The project supports **SDG 9 – Industry, Innovation and Infrastructure** by applying software engineering and automation concepts to modern industrial infrastructure.

E.8 CONCLUSION

The proposed Java-based Warehouse Management System successfully demonstrates how object-oriented programming can be applied to an autonomous warehouse environment.

The system models robots, items, locations and tasks as objects. Encapsulation protects internal data, inheritance provides reusable robot functionality, and polymorphism allows different robot types to execute tasks according to their specialized behaviour.

The Java Collection Framework provides efficient storage and retrieval of warehouse objects, while custom exception handling improves reliability. Multithreading, synchronization and thread priority demonstrate how warehouse tasks can be processed concurrently.

The architecture is modular and extensible, allowing new robot types and operations to be introduced with minimal changes. Therefore, the proposed design provides a suitable foundation for a larger autonomous warehouse management application.

E.9 STUDENT REFLECTION

This assignment helped in understanding how theoretical Java concepts can be applied to a practical software engineering problem.

The major learning outcomes were:

- Designing classes based on real-world entities.
- Understanding the importance of encapsulation.
- Applying inheritance and polymorphism.
- Using Java collections with generics and iterators.
- Handling application-level exceptions.
- Understanding threads and synchronization.
- Designing software for future extensibility.

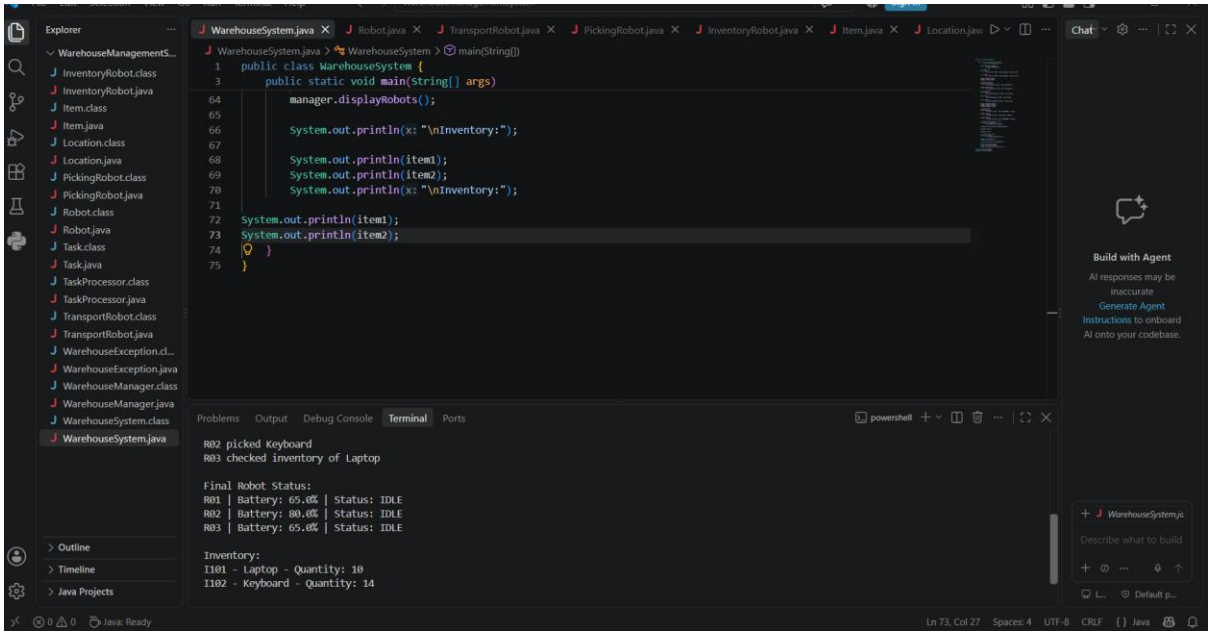
The assignment also improved understanding of how individual Java concepts work together to create a complete software system.

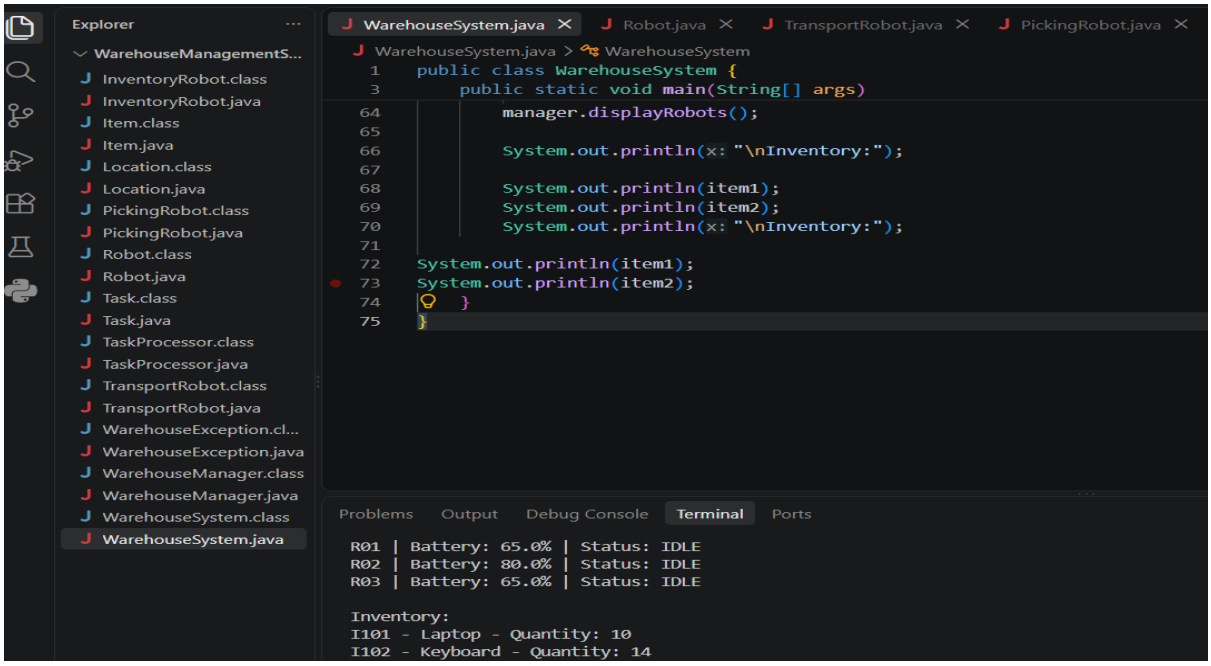
With additional resources, the project could be improved by integrating a database, graphical interface, REST APIs, real-time robot tracking and IoT sensors.

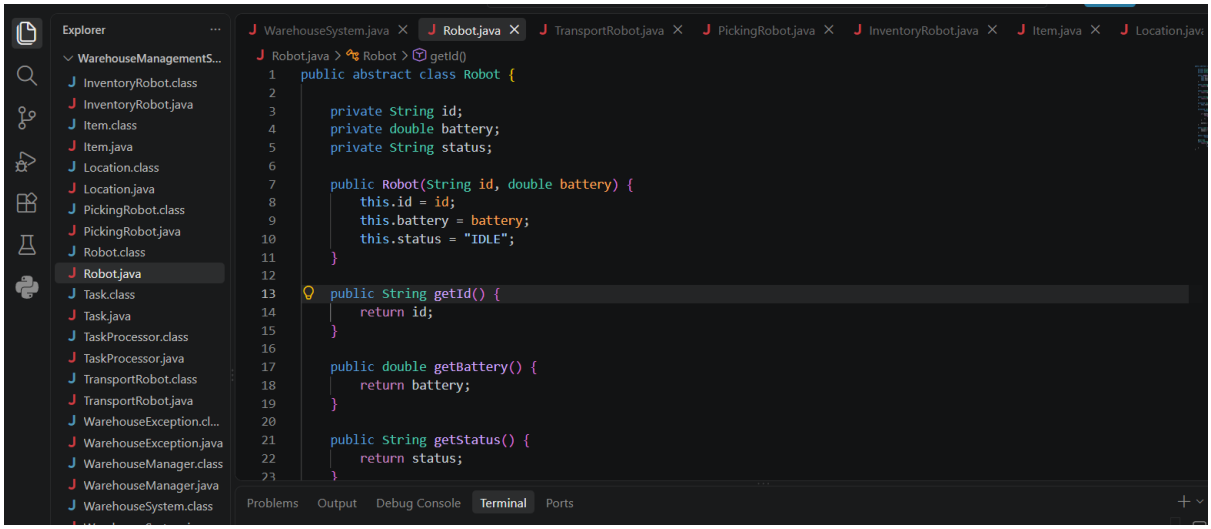
E.10 REFERENCES

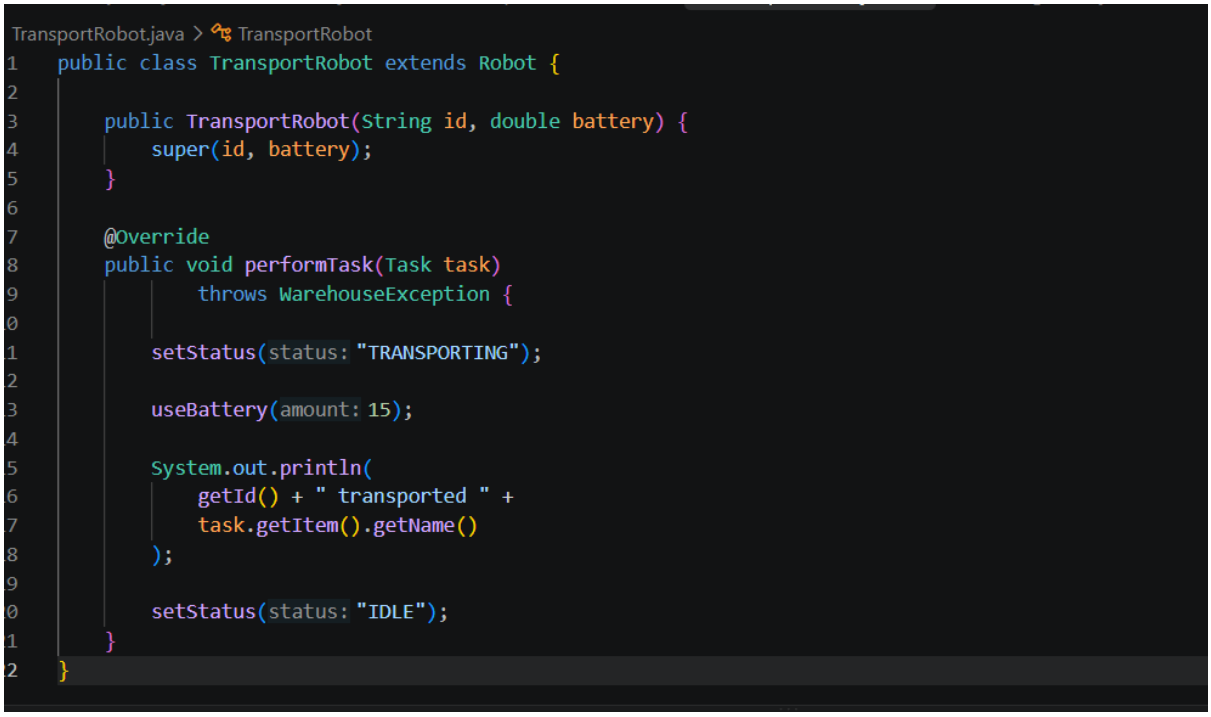
1. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill.
2. Cay S. Horstmann, *Core Java*, Pearson.
3. Kathy Sierra and Bert Bates, *Head First Java*, O'Reilly Media.
4. Oracle, *Java Documentation*.
5. Oracle, *Java Collections Framework Documentation*.
6. Oracle, *Java Concurrency and Multithreading Documentation*.
7. Course material provided for **CSA09 – Programming in JAVA**.

Implementation


Explorer
WarehouseManagementS...
InventoryRobot.class
InventoryRobot.java
Item.class
Item.java
Location.class
Location.java
PickingRobot.class
PickingRobot.java
Robot.class
Robot.java
Task.class
Task.java
TaskProcessor.class
TaskProcessor.java
TransportRobot.class
TransportRobot.java
WarehouseException.cl...
WarehouseException.java
WarehouseManager.class
WarehouseManager.java
WarehouseSystem.class
WarehouseSystem.java
Outline
Timeline
Java Projects
WarehouseSystem.java
Robot.java
TransportRobot.java
PickingRobot.java
InventoryRobot.java
Item.java
Location.java
WarehouseSystem.java > WarehouseSystem > main(String[])
1 public class WarehouseSystem {
3 public static void main(String[] args)
64 manager.displayRobots();
66 System.out.println(x: "\nInventory:");
68 System.out.println(item1);
69 System.out.println(item2);
70 System.out.println(x: "\nInventory:");
72 System.out.println(item1);
73 System.out.println(item2);
74 }
75 }
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Problems Output Debug Console Terminal Ports
powershell
R02 picked Keyboard
R03 checked inventory of Laptop
Final Robot Status:
R01 | Battery: 65.0% | Status: IDLE
R02 | Battery: 80.0% | Status: IDLE
R03 | Battery: 65.0% | Status: IDLE
Inventory:
I101 - Laptop - Quantity: 10
I102 - Keyboard - Quantity: 14
Describe what to build
Default p...
Java: Ready
Ln 73, Col 27 Spaces: 4 UTF-8 CRLF { } Java


Explorer
WarehouseManagementS...
InventoryRobot.class
InventoryRobot.java
Item.class
Item.java
Location.class
Location.java
PickingRobot.class
PickingRobot.java
Robot.class
Robot.java
Task.class
Task.java
TaskProcessor.class
TaskProcessor.java
TransportRobot.class
TransportRobot.java
WarehouseException.cl...
WarehouseException.java
WarehouseManager.class
WarehouseManager.java
WarehouseSystem.class
WarehouseSystem.java
WarehouseSystem.java
Robot.java
TransportRobot.java
PickingRobot.java
WarehouseSystem.java > WarehouseSystem
1 public class WarehouseSystem {
3 public static void main(String[] args)
64 manager.displayRobots();
66 System.out.println(x: "\nInventory:");
68 System.out.println(item1);
69 System.out.println(item2);
70 System.out.println(x: "\nInventory:");
72 System.out.println(item1);
73 System.out.println(item2);
74 }
75 }
Problems Output Debug Console Terminal Ports
R01 | Battery: 65.0% | Status: IDLE
R02 | Battery: 80.0% | Status: IDLE
R03 | Battery: 65.0% | Status: IDLE
Inventory:
I101 - Laptop - Quantity: 10
I102 - Keyboard - Quantity: 14


Explorer
WarehouseManagementS...
InventoryRobot.class
InventoryRobot.java
Item.class
Item.java
Location.class
Location.java
PickingRobot.class
PickingRobot.java
Robot.class
Robot.java
Task.class
Task.java
TaskProcessor.class
TaskProcessor.java
TransportRobot.class
TransportRobot.java
WarehouseException.cl...
WarehouseException.java
WarehouseManager.class
WarehouseManager.java
WarehouseSystem.class
WarehouseSystem.java
Robot.java
TransportRobot.java
PickingRobot.java
InventoryRobot.java
Item.java
Location.java
Robot.java > Robot > getId()
public abstract class Robot {
private String id;
private double battery;
private String status;
public Robot(String id, double battery) {
this.id = id;
this.battery = battery;
this.status = "IDLE";
}
public String getId() {
return id;
}
public double getBattery() {
return battery;
}
public String getStatus() {
return status;
Problems
Output
Debug Console
Terminal
Ports


TransportRobot.java > TransportRobot
public class TransportRobot extends Robot {
public TransportRobot(String id, double battery) {
super(id, battery);
}
@Override
public void performTask(Task task)
throws WarehouseException {
setStatus(status: "TRANSPORTING");
useBattery(amount: 15);
System.out.println(
getId() + " transported " +
task.getItem().getName()
);
setStatus(status: "IDLE");
}
}